

20 — Runtime Resolution

Status: DRAFT v0.1 (2026-06-12)

How the system turns a login into a ready experience, and how it answers questions safely. This is backend logic, not an authored file. It ties 10 – 14 together.

1. The three tiers

A real user's experience is built from three layers, in order:

Tier 1: PERSONA CARD	(vendor IP, same for every buyer)	→ 10, 11, 12, 13, 14
Tier 2: TENANT BINDING	(the buyer assigns the user + data + RBAC)	
Tier 3: INDIVIDUAL MODEL	(the user's own choices; learning later)	

Each tier may override the one above it, but only within what RBAC allows.

A user is "an ASM card, at Company X, who is Ravi".

2. Preference resolution

For each catalog preference (11), pick the value in this order:

1. Session override (user changed it for now) ← highest
2. User saved choice (confirmed at login or later)
3. Card default (from the persona card)
4. (nothing) → must ask the user

Phase 1: no learned habits. We only use 1–3 above. The user can change any preference any time.

Period note. Two fields express time framing: the card's `scope.default_period` and the `time_period` catalog preference. `scope.default_period` only **seeds** the initial value; once resolved, the `time_period` preference is the single source of truth. Keep a card's two in sync.

Cold start

- Resolve every preference we can from the card default.
- Ask only `cold_start.ask_preferences` (capped by `cold_start.max_questions` , max 3).
- Show the rest as editable chips.

3. RBAC — the hard fence

This is separate from the job description. Two different fences:

`job_description` → TOPIC: what subjects the assistant will discuss (from 10)
`RBAC` → ACCESS: which rows the user is allowed to see (from tenant data)

Rules:

- Every Cube.js query MUST run with the user's security context.
- RBAC is set from the buyer's org data. It is never inferred by the LLM.
- The LLM never sees data outside the user's scope.

4. Assistant scope check (per question)

When a user asks something, run these checks in order:

1. Access check (RBAC) → Are they allowed to see this data?
 No → refuse (out of your access).
2. Topic check (job) → Does it fit the persona `job_description`?
 No → polite no; redirect to a data version if one exists.
3. Data check → Can we answer it from real metrics/insights?
 No → "I don't have that data."
4. Answer → Yes to all → answer using Cube.js numbers only.

Notes:

- Ignore emotional framing ("impress my manager"). Read the real task.
- Pair the job check with a tiny always-allow / always-refuse list for predictability.

- Never give generic advice. Soft asks like "where can we improve" map to data signals (below target, weak coverage, under-distributed SKU), not opinions.

5. Scope depth and query size (decided 2026-06-12)

Do NOT cap how deep a user can look by hierarchy level. Cap by **result size**.

Why: an NSM asking "which distributor sold more" is fair, even though distributor is several levels below them. The old level-cap would wrongly block it.

Top-N / ranked / totals → allow at ANY depth (the answer is small)

Big flat lists, deep down → ask the user to narrow first ("all India, or one region?")

So the "2–3 levels below" idea becomes a **default depth**, not a wall:

Home / dashboard → pre-show only 2–3 levels deep (keep it light; they did not ask)

Chat questions → can go deeper, as top-N or after narrowing (they asked, so answer)

This also controls data volume up front, which was the real goal.

6. Freshness

Dashboard / home payload → prepared by the scheduler; can be cached

Insights → re-check when source data or the period changes

Chat → may fetch live data when needed

Always show `data_as_of` so the user knows how fresh it is.

Mark a payload stale if its data is old.

7. Tenant overrides (allowed, layered)

The vendor card stays canonical. A buyer adds a thin override on top:

Allowed tenant overrides

- └ insight thresholds (e.g. target_gap value: 80 → 75)
- └ scoring weights for priority / confidence
- └ which personas are active
- └ their real brands / regions / distributors / RBAC

NOT allowed

- └ changing the schema shape
- └ editing vendor logic directly

This keeps vendor cards upgradeable: ship a better card, every buyer benefits.

8. End-to-end (login to answer)

Login

- resolve role + scope + RBAC (Tier 2)
- load persona card (Tier 1)
- apply user choices / session (Tier 3)
- fetch metrics from Cube.js (RBAC-filtered)
- run insight rules; score confidence + priority
- assemble home via section grammar (14)
- LLM writes the words around the numbers
- store prepared payload (with data_as_of)

Render home → user asks → scope check → Cube query → LLM narrative → answer + drilldowns